

Smart Breadboard Diagnostic Trainer

A web replacement for the hardware breadboard trainer. Students build a circuit in a 3D workspace; the system extracts the netlist they actually built, sets it against the instructor's captured reference, and names the difference — which pin, which row, which rail. Two documents follow: one written for a technical reader, one for a general one.

DOCUMENT

Version 1.0

DATE

September 2026

DOMAIN

Electronics laboratory
instruction

BUILD STATUS

Phases 0–5 complete

.....

Document 2 · General overview

No technical background assumed. For readers outside electronics and computing.

01

TITLE

Smart Breadboard Diagnostic Trainer: a practice laboratory for electronics students that explains what went wrong

Short title: Breadboard Trainer.

The problem, in plain terms

Electronics students learn by building small circuits — a battery, a light, a few components, connected by wires. They build them on a **breadboard**: a small plastic board covered in tiny holes, where pushing a wire into a hole connects it to whatever else is in the same row. It is the electronics equivalent of a practice sketchpad. Nothing is glued or soldered, so a circuit can be built, taken apart and built again.

The problem is what happens when the circuit does not work.

A wrong circuit does not announce itself. The light simply does not turn on. There is no error message, no red underline, no hint. The student is left staring at a board of forty wires with no idea which one is the mistake — and the mistake is usually tiny: a wire one hole too far to the left, a component inserted backwards, a connection that was never made at all.

So the student raises a hand and waits. The instructor, who has thirty to forty other students in the same two-hour session, eventually arrives, glances at the board and points at the wire. The student fixes it and moves on.

Nobody learned anything from that exchange. The student learned that asking works. What they were supposed to learn — how to find the mistake themselves — is the one thing the session never taught, because there was no way to practise it.

What this system does

The Smart Breadboard Diagnostic Trainer moves the practice board into the computer, and gives it the one thing the real board cannot have: **the ability to explain itself**.

A student opens the exercise in a web browser and sees a realistic breadboard in 3D, which can be rotated and zoomed. They drag components onto it and push wires into holes, in the same way and in the same order as they would on the real thing.

Behind the scenes, the system is not watching what the board *looks* like. It is tracking what is electrically connected to what — the same understanding an experienced instructor has when they glance at a board and immediately see the problem.

The instructor has already built the correct version of that circuit once, inside the same program. The system saved it, not as a picture, but as a record of which connections the correct circuit has. When a student builds their version, the system compares the two records and finds the difference.

If there is a difference, the system does not say *wrong* and it does not give a score. It highlights the exact row on the board where the problem is, and it says something like this:

⚠ BROKEN CONNECTION

The LED never reaches ground. The jumper wire is on the far side of the centre channel, so row 14 and row 15 are two separate connections rather than one.

Which side of the channel does that wire need to be on?

That question is the point of the whole system. The student is told exactly where to look, and then asked to work out the fix themselves. If they cannot, they can ask for a stronger hint, and then for the answer outright — but each step is a deliberate choice, and the system records how far they had to go.

Why the comparison is the clever part

There is one problem that makes this harder than it sounds, and solving it is the technical contribution of this project.

There is no single correct-looking circuit. The same circuit can be built on row 5 or row 30, with the components arranged differently, and be completely and equally correct. Two correct boards can look nothing alike.

A system that compared pictures would fail immediately — it would mark a perfectly good circuit as wrong simply because the student built it further down the board. Students would stop trusting it within one session, and a tool nobody trusts is worse than no tool at all.

This system compares **connections**, not appearances. It asks “is this component's leg connected to the same things it should be connected to?” rather than “is it in the same place?” Two circuits built in completely different positions on the board come out identical, because electrically they *are* identical.

That is why the system can be confident enough to point at a specific wire and say *that one* — and it is why the project sets itself a strict measurable target: when a student builds the circuit correctly but differently, the system must report no problems at all, at least 95% of the time.

What the instructor gets

The instructor sets up an exercise once and lets it run. They build the correct circuit themselves — no writing of specifications, no lists of rules, just building it the way they would on their own bench. The system remembers it and uses it to check every student in the class.

While students work, the system handles the ordinary mistakes that account for most raised hands: the missing wire, the backwards component, the connection in the wrong row. That leaves the instructor free to spend their limited time on the students who are genuinely stuck on the concept rather than on the wiring.

Afterwards, the instructor gets a summary of the whole class: which mistakes came up most often, how long students took to find their own errors, how many worked it out without asking for help, and which students are struggling and have not said so. That summary can be exported to a spreadsheet.

None of this replaces the instructor's judgement. The system diagnoses; the grade stays with the teacher.

For the student

FEATURE	WHAT IT MEANS IN PRACTICE
Realistic 3D practice board	A full-size breadboard that can be rotated and zoomed, with components dragged into place. It behaves like the real thing, including refusing impossible placements.
Nothing is graded while you build	The system stays quiet while the student is still working. It speaks up only once a section is finished, or once a wire has been left dangling for several seconds. It is not designed to nag.
Plain-language explanations	Mistakes are described in the language of the board — which wire, which row, which component — never as a percentage or a mark.
Guiding questions before answers	The first response to a mistake is a question, not a correction. A stronger hint and then the explicit answer are available on request, one deliberate step at a time.
Visual pointing	The row containing the mistake glows on the board while the rest dims, and a button swings the camera round to show it.
Free practice	Students can practise outside laboratory hours, as often as they like. Nothing is consumed and nothing can be damaged.
Work is saved automatically	Closing the browser mid-build loses nothing. Reopening the exercise restores the board exactly as it was left.
Works without internet	Once an exercise has been opened once, it can be completed with the connection down. Everything syncs when the network returns.
English and Filipino	The whole interface is available in both. Technical words such as <i>resistor</i> and <i>breadboard</i> stay in English on purpose, because that is how the subject is taught and examined.
Built for different eyes and hands	Larger text, higher contrast, and reduced motion for anyone who finds animation uncomfortable. Everything works by keyboard alone. Crucially, no problem is ever shown by colour alone — every warning carries a symbol and words as well, so colour-blind students, roughly one man in twelve, are never left guessing.
Installable like an app	It can be added to a phone or laptop home screen and opened like any other application.

For the instructor

FEATURE	WHAT IT MEANS IN PRACTICE
Class setup	Create a class, and the system generates a six-character code. Students type that code once to join.
Class list management	See who has joined, remove a student, or issue a new code.
Exercise authoring	Set the title, the objective, the difficulty, the list of parts allowed, and an optional schematic diagram. Exercises stay hidden from students until deliberately published.
Build the answer once	The correct circuit is defined by building it, not by describing it. The system reads it, shows the instructor what it understood, and saves it as the reference for that exercise.
Validation on saving	If the reference circuit itself has a problem — a loose connection, a short, an unused component — the system refuses to save it and says why. It will not let a broken answer key be published.
Class analytics	The most common mistakes, the average time students take to find their own errors, how many resolved a problem without asking for help, and a list of students who appear to be struggling.
Individual review	A timeline of a single student's attempt, showing what they built, what went wrong, and what they did about it.
Spreadsheet export	All of it exports to a standard spreadsheet file.

Protecting the answer key

There is one thing that would break this system completely: if a student could get hold of the instructor's reference circuit, every exercise would become a copying task rather than a learning one.

This was treated as the central security concern of the project rather than an afterthought, and it is guarded in four independent ways. The reference circuit is filtered out of the information sent to students at four separate points, each one an independent check, so that a mistake at any single point is caught by the others. An automated test runs on every change to the software and inspects the actual program delivered to student browsers, confirming that no trace of the answer key is present. **That test has already caught two genuine leaks during development**, before either one could reach a student.

Related protections:

- Nobody can create a teacher account by simply choosing “teacher” when signing up. A teacher account requires a code issued by the department, and if no code has been

configured, no teacher account can be created at all. Without this, any student could see every answer key just by ticking the wrong box at registration.

- One student cannot open another student's work.
- Passwords are never stored in a readable form, and no login information is ever kept anywhere a malicious web page could reach it.
- Repeated failed sign-in attempts are slowed down automatically.
- Password reset links can only be used once, and they expire.

Proving that it works

A capstone project should not merely claim its system is accurate. This one is built to demonstrate it.

A test collection of **150 deliberately broken circuits** has been prepared, each labelled with the mistake it actually contains. The system is run against all 150 automatically, and its diagnosis is compared with the known answer. It reports how often it pointed at the right place, how often it named the right kind of mistake, and how often it complained about a circuit that was in fact correct.

Twenty of those cases are correct circuits deliberately built in unusual positions. The system must find nothing wrong with any of them. These tests run automatically every time the software changes, so accuracy is re-measured continuously and cannot quietly get worse.

Points at the right place	at least 90%
Names the right kind of mistake	at least 85%
Wrongly flags a correct circuit	no more than 5%

The electronics student — the main user

A second- or third-year college student taking an electronics laboratory course, usually 18 to 21 years old, using a school computer or an ordinary laptop. They are comfortable with a web browser and do not need to know anything about programming.

Right now, their laboratory experience has a gap in it. They are graded on whether the circuit works, but they are never actually taught the skill that decides whether it works: methodically finding the one connection that is wrong. That skill is learned by trying, failing, being told precisely what went wrong, and trying again — a cycle the current laboratory setup makes almost impossible, because the board says nothing and the instructor is busy.

This system gives them that cycle. It can be repeated as many times as they want, outside class hours, without waiting for anyone and without embarrassment. And because the system asks a question before it gives an answer, the student is still the one doing the finding.

The design also assumes some realities of their situation: campus internet that drops out, shared laboratory computers that reset overnight, a bilingual classroom, and the ordinary fact that some of them cannot reliably distinguish red from green.

The laboratory instructor — the second main user

A faculty member responsible for one or more laboratory sections of thirty to forty students, in sessions of about two hours.

Their difficulty is arithmetic rather than pedagogical. Forty boards cannot be inspected by one person in two hours. Most of a session is spent queuing, and most of the instructor's attention goes to mistakes that are trivial to spot and tedious to keep spotting.

This system takes the repetitive half of that work. It answers the routine questions immediately and consistently, so the instructor's time goes to the students who are stuck on the idea rather than the wiring. It also tells them things they currently have no way of knowing: which mistakes this particular class keeps making, which exercise is harder than intended, and which quiet student has been stuck for forty minutes without saying so.

The department — the indirect beneficiary

A laboratory section costs money in components, and components get damaged, lost and used up. A practice board that lives in a browser costs nothing per additional student, cannot be broken, and is available at any hour. It does not replace the physical laboratory — students still need to handle real hardware — but it lets them arrive at the real bench already knowing what they are doing.

What this system deliberately does not do

Worth stating clearly, so that expectations are accurate.

- It is **not** a professional electronics design program. It is a teaching board for classroom circuits, and nothing more.
- It handles **steady, direct-current circuits only** — the kind used in introductory laboratory work. Circuits involving alternating current or rapidly changing signals are outside its scope, and that limit is a deliberate decision rather than an omission.
- It does **not** assign grades. It explains mistakes; the marking stays with the instructor.
- It does **not** replace the physical laboratory. Students still need to touch real components. This is preparation and practice, not a substitute.
- It does **not** use artificial intelligence to work out what is wrong. Every diagnosis comes from fixed, tested electrical rules. This matters for the research: the system gives the same answer for the same circuit every single time, which is what makes its accuracy measurable and the results of this study repeatable by anyone who checks them.

Current status of the project

Stated plainly, so that no part of this document is read as a claim about something that is not yet built.

Built and working today

The complete student-facing application exists as a working program: account creation with email confirmation, secure sign-in, password reset, first-time setup, joining a class by code, the student dashboard with the full exercise library, the exercise briefing screen, the settings screen with its language and accessibility options, and the full visual design system underneath all of it. Both languages are complete. The protections around the answer key and around teacher accounts are in place and independently tested.

Verified by automated testing

The program is checked by **128 automated tests** running against a real database, plus **23 tests driving a real web browser** from account creation through to signing out. All of them pass. These are not manual checks — they run automatically and would fail loudly if anything broke.

Still to be built

The 3D building workspace itself, the circuit reading and comparison engine, Learn Mode for instructors, the class analytics, and the offline capability. These are the remaining planned phases of the project, and each has a written specification and a defined completion checklist.

Waiting on accounts rather than on work

Three items are finished as code but cannot be ticked off yet because they need online accounts that have not been set up — the hosted database, the web hosting service and the code repository. None of them requires further development.

How the project will be judged

Two separate measurements, taken independently, and a third that asks whether it helps.

Does it diagnose correctly? Measured automatically against the 150 prepared test circuits, and reported as three percentages: how often it pointed at the right place, how often it named the right kind of mistake, and how often it wrongly flagged a correct circuit.

Is it a good product? Measured using **ISO/IEC 25010**, an international standard for judging software quality across five areas — whether it does what it should, whether it performs well, whether people can actually use it, whether it is dependable, and whether it runs on the machines it needs to. The target is an average rating of **4.20 out of 5**.

Does it actually help? A trial with at least 30 students runs the same exercises two ways: one group with the explanations turned on, and one group told only whether their circuit passed or failed. Comparing how long each group takes to find their own mistakes is what shows whether the system teaches, rather than merely functions.

Smart Breadboard Diagnostic Trainer · capstone system documentation, version 1.0 · September 2026.
Source documents: docs/system-documentation-technical.md and docs/system-documentation-general.md.